TOKENDECK: MONITOR DE MESA PARA COTAS DE USO DE ASSISTENTES DE IA COM ESP32

Gustavo da Cunha Constante, João Victor Nunes Peruchi, João Marcos Vieira dos Santos, Brayan Miguel Favarin, Bruno Luque Rego
gustavo.cunha.constante@unesc.net, joaovnperuchi@gmail.com, joaomarcos.vieirasantos@unesc.net, brayanmf1227@unesc.net, itzluqueezv17@unesc.net

Internet das Coisas – Valter Blauth Junior
valterblauth@unesc.net

Resumo: Este trabalho apresenta o TokenDeck, um sistema de monitoramento das cotas de uso de assistentes de inteligência artificial para programação (Codex, Claude Code e Gemini) formado por um monitor físico de mesa, baseado na placa ESP32-2432S028R (CYD 2.8"), e por uma aplicação desktop para Windows. Uma ponte escrita em Python consulta periodicamente as janelas de uso de 5 horas e semanal de cada assistente, converte as medidas em mensagens JSON e as envia por porta serial USB à placa, que as exibe em uma interface gráfica LVGL e troca de tela automaticamente conforme o assistente em uso. O firmware, em C++ sobre Arduino-ESP32, valida os dados recebidos, persiste o brilho da tela em memória não volátil e conta com watchdog de hardware; a aplicação desktop reúne o painel, a configuração, o histórico de sete dias com projeção de consumo e um assistente de instalação. Uma suíte de 130 testes automatizados cobre a ponte, a leitura dos provedores e a persistência. O protótipo atualiza o painel físico e a aplicação a cada 5 segundos sem acessar o conteúdo das conversas, o que o torna viável como instrumento de supervisão de baixo custo.
Palavras-chaves: Internet das Coisas. ESP32. Monitoramento de cotas. Comunicação serial.

1. Introdução

Os assistentes de inteligência artificial para programação tornaram-se parte do cotidiano de quem desenvolve software. Em planos por assinatura, o uso desses serviços é limitado por janelas de tempo: em geral uma janela curta, de 5 horas, e um limite semanal. Esgotar a cota no meio de uma tarefa interrompe o trabalho, e cada ferramenta mostra essa informação em um lugar diferente, como um comando do terminal, uma tela de configurações ou um arquivo de registro, o que obriga o desenvolvedor a consultas manuais frequentes.

Em paralelo, a Internet das Coisas (IoT) consolidou-se como o paradigma em que objetos dotados de capacidade de sensoriamento, processamento e comunicação cooperam para atingir um objetivo comum (ATZORI; IERA; MORABITO, 2010; GUBBI et al., 2013). Essa visão dialoga com a computação ubíqua proposta por Weiser (1991), na qual a informação fica na periferia da atenção do usuário e pode ser percebida de relance, sem exigir a abertura de uma aplicação.

Portanto, este trabalho tem como objetivo desenvolver o TokenDeck, um protótipo que reúne em um único ponto o consumo das cotas de três assistentes (Codex, Claude Code e Gemini) e o apresenta, em tempo real, em um monitor físico de baixo custo colocado ao lado do computador e em uma aplicação desktop. O diferencial do protótipo está em unificar fontes heterogêneas em um formato único de mensagem e em fazê-lo sem ler, armazenar ou transmitir prompts, respostas ou chaves privadas.

O artigo está organizado da seguinte forma: a seção 2 descreve as cotas de uso e suas fontes de dados; a seção 3 apresenta os conceitos de IoT e a plataforma de prototipação; a seção 4 detalha a arquitetura; a seção 5 trata da implementação; a seção 6 descreve a metodologia e os testes; a seção 7 discute os resultados; e a seção 8 traz as considerações finais.

2. Cotas de uso de assistentes de IA

Os três assistentes considerados expõem o consumo de forma semelhante em conceito, mas diferente na prática. Em todos eles há uma janela de curta duração, de 5 horas, e uma janela semanal, e para cada uma delas é possível conhecer a fração já consumida e o instante do próximo reinício. As fontes utilizadas foram:

- Codex (OpenAI): registros locais das sessões, em formato JSONL, que trazem os campos de limite de taxa da conta;
- Claude Code (Anthropic): cabeçalhos de limite de taxa devolvidos pela API em uma requisição mínima, de um único token de saída (ANTHROPIC, 2026);
- Gemini (Google): comando de uso da CLI Antigravity, executado de forma não interativa e com saída em JSON.

Como cada fonte usa nomes, escalas e formatos de data distintos, o sistema normaliza os dados para quatro grandezas: percentual consumido na janela de 5 horas, minutos até o seu reinício e os equivalentes para a janela semanal. Essa normalização é o que permite que um único firmware e uma única interface exibam os três provedores.

Vale registrar que os cabeçalhos de limite usados no caso do Claude Code não fazem parte de uma interface formalmente documentada, de modo que o código trata a ausência ou a mudança desses campos como falha de leitura e não como erro fatal.

3. Internet das Coisas e plataforma de prototipação

A IoT costuma ser descrita por três pilares: comunicação, controle e automação, e redução de custos. No TokenDeck a comunicação ocorre entre a ponte e a placa por meio de mensagens JSON; o controle é exercido pelo usuário ao ajustar o brilho, ligar ou desligar provedores e escolher o

formato do relógio; e a redução de custos vem do uso de uma placa de prateleira em vez de um equipamento dedicado. Os “sensores” do sistema são virtuais: os próprios provedores, que medem o consumo.

A plataforma de prototipação escolhida foi a ESP32-2432S028R, conhecida como CYD (*Cheap Yellow Display*) 2.8". Ela integra um microcontrolador ESP32 de dois núcleos (ESPRESSIF, 2023), uma tela TFT de 320×240 pixels com controlador ILI9341 (ILITEK, 2011), painel resistivo sensível ao toque XPT2046 e um conversor USB-serial CH340. A placa não possui PMU, sensor inercial nem memória PSRAM, restrição que influenciou várias decisões de projeto.

O firmware foi construído com o PlatformIO, sobre o framework Arduino para ESP32 (versão 2.x), usando a biblioteca gráfica LVGL 9 para a interface, a GFX Library for Arduino como driver da tela e a ArduinoJson 7 para a interpretação das mensagens (LVGL, 2024; PLATFORMIO, 2024; BLANCHON, 2024). A aplicação desktop usa Python, a biblioteca pyserial para o enlace com a placa (PYSERIAL, 2024) e o pywebview para hospedar uma interface em HTML, CSS e JavaScript sobre o WebView2 do Windows (PYWEBVIEW, 2024).

Versões anteriores do projeto comunicavam-se com o computador por Bluetooth de baixa energia (BLE). Esse caminho foi abandonado em favor da conexão por cabo USB, que dispensa o pareamento, alimenta a placa e mantém as credenciais dos provedores restritas ao computador, sem que trafeguem por uma rede sem fio. Uma alternativa por Wi-Fi também foi descartada pelas mesmas razões.

4. Arquitetura do sistema

A Figura 1 resume o fluxo de dados. Um processo único, a ponte, coleta as informações dos três provedores, normaliza-as e as distribui para dois consumidores: a placa CYD, por porta serial USB a 115200 bit/s, e a aplicação TokenDeck Studio, por arquivos JSON locais com o último estado e o histórico de amostras.

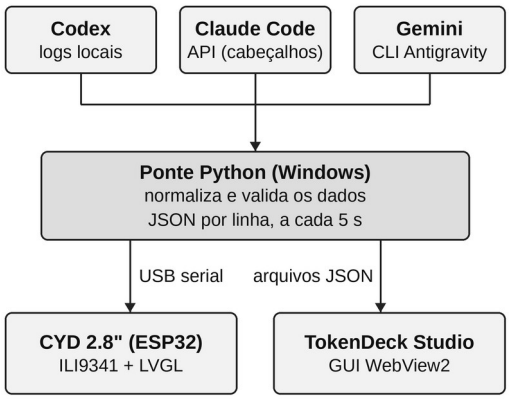


Figura 1 – Arquitetura do TokenDeck.
Fonte: Os autores (2026).

O protocolo é textual: a cada ciclo a ponte envia uma linha JSON por provedor, terminada em quebra de linha. A Tabela 1 lista os campos. A escolha do JSON (BRAY, 2017) simplifica a depuração, pois a comunicação pode ser observada em qualquer monitor serial, e permite acrescentar campos opcionais sem quebrar firmwares mais antigos, já que campos ausentes assumem valores padrão.

Tabela 1 – Campos da mensagem enviada à placa.

Campo	Significado
`p`	provedor: codex, claude ou gemini
`s, sr`	uso (%) da janela de 5 h e minutos até o reinício
`w, wr`	uso (%) da janela semanal e minutos até o reinício
`st`	estado: allowed, limited, unavailable ou disabled
`ok`	indica leitura válida
`a`	assistente com a atividade local mais recente
`brt`	brilho da tela (5 a 255)
`t, tf`	hora local e formato do relógio (12 ou 24 h)

Fonte: Os autores (2026).

5. Implementação

Firmware. O código da placa contém cerca de 3,4 mil linhas de C++ e segue uma camada de abstração de hardware: tela, toque, entrada, energia e sensor inercial são interfaces que cada placa implementa em sua própria pasta, de modo que a interface gráfica e o laço principal não contêm código específico de placa. Portar o sistema para outro hardware consiste em adicionar uma pasta e um ambiente de compilação. No CYD, a aplicação ocupa cerca de 1,6 MB de flash, motivo pelo qual usa a tabela de partições ampla, sem atualização pelo ar.

A interface mostra, para o provedor ativo, as barras de consumo das janelas de 5 horas e semanal, o tempo restante até o reinício e o estado da conta. Quando a ponte informa o campo de atividade, a tela passa automaticamente para o assistente que o usuário acabou de utilizar. Como a placa não tem PSRAM, os recursos gráficos foram reduzidos: sem captura de tela, sem animações quadro a quadro e com um ícone estático no lugar do mascote animado das placas maiores.

Por receber dados de um processo externo, o firmware trata a entrada como não confiável: percentuais são limitados ao intervalo de 0 a 100, o formato do relógio cai para 24 h quando o valor é desconhecido, e linhas maiores que o buffer de 512 bytes são descartadas em vez de interpretadas pela metade. O brilho recebido (5 a 255) é gravado na memória não volátil (NVS) e restaurado na partida, e um watchdog de hardware de 10 segundos reinicia a placa caso o laço principal trave.

Ponte. A ponte detecta a porta da placa pelo identificador USB do CH340, sem jamais escolher as portas internas do computador, e abre a conexão com as linhas DTR e RTS inativas, pois em muitas placas elas estão ligadas aos pinos de reinício e de boot do ESP32 e a simples abertura da porta reiniciaria o microcontrolador. Em caso de desconexão, ela volta ao ciclo de busca e reconecta sozinha; o número de reconexões é registrado e mostrado na aplicação, como sinal precoce de cabo ou porta instável em um equipamento que opera sem supervisão.

O ciclo de atualização é de 5 segundos. O Codex é lido do disco a cada ciclo, enquanto o Claude Code e o Gemini, que dependem de chamadas externas, são consultados a cada 60 segundos. A atividade local é detectada pelos horários de modificação dos

arquivos de cada ferramenta e, no caso do Codex, pelo horário da última mensagem do usuário, sem que o conteúdo seja armazenado. A cada 5 minutos uma amostra é anexada ao arquivo de histórico, com bloqueio de arquivo para evitar gravações concorrentes entre a ponte e a aplicação.

Aplicação desktop. O TokenDeck Studio possui quatro abas: Painel, com os três provedores lado a lado e um modo de foco ao passar o ponteiro; Config, com o brilho, o relógio e a habilitação de cada provedor; Histórico, com a tendência dos últimos sete dias, o resumo diário (pico, média e reinícios detectados), a projeção semanal de consumo e a exportação dos dados brutos; e Setup, um assistente de quatro passos para o primeiro uso. A aplicação também inicia e encerra a ponte e grava o firmware na placa, liberando antes a porta serial. A mudança de brilho feita no controle deslizante chega à placa em cerca de 0,3 s, pois a espera entre ciclos é dividida em fatias curtas.

6. Metodologia e testes

O desenvolvimento foi iterativo: a cada etapa uma funcionalidade era implementada, observada na placa real e refinada. A equipe de cinco integrantes dividiu o trabalho da seguinte forma: Gustavo da Cunha Constante, líder, responsável por hardware, software e documentação; João Victor Nunes Peruchi, João Marcos Vieira dos Santos e Brayan Miguel Favarin, responsáveis por software; e Bruno Luque Rego, responsável pela apresentação.

Como a interface da placa não pode ser capturada por software no CYD, as alterações visuais foram verificadas diretamente no equipamento. Já a ponte e a leitura dos provedores contam com uma suíte automatizada de 130 testes em pytest, que cobre a interpretação da saída da CLI do Gemini, a leitura dos registros do Codex, a obtenção do token, a detecção de atividade, a reconexão, a persistência do histórico e a criação de atalhos e inicialização automática no Windows.

7. Resultados e discussão

A Figura 2 mostra o monitor físico em funcionamento, exibindo a tela de uso do Codex, com 55% da janela de 5 horas e 24% da janela semanal consumidos e o tempo restante até cada reinício. A Figura 3 mostra o painel do TokenDeck Studio com os três provedores e o indicador de conexão da placa na porta serial.



Figura 2 – Monitor físico (CYD 2.8") em operação. Fonte: Os autores (2026).

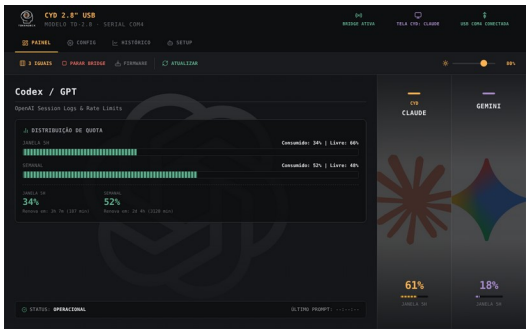


Figura 3 – Painel do TokenDeck Studio.
Fonte: Os autores (2026).

O protótipo cumpriu o objetivo de concentrar em um só lugar as cotas dos três assistentes. A placa e a aplicação recebem a mesma informação, de modo que o usuário pode optar entre a consulta de relance, no monitor de mesa, e a consulta detalhada, na aplicação. O monitor físico atende bem ao caso de uso de tecnologia calma: a barra de progresso e o tempo até o reinício podem ser lidos sem interromper a tarefa em andamento.

A decisão de manter a comunicação por cabo mostrou-se adequada ao problema, pois simplificou a instalação e eliminou uma classe de falhas de pareamento. Em contrapartida, ela impede o uso do monitor sem o computador e prende a solução ao sistema operacional Windows, já que a ponte, a detecção de instâncias e os atalhos usam recursos específicos desse sistema.

A principal fragilidade identificada está nas fontes de dados. A leitura do Claude Code depende de cabeçalhos não documentados formalmente e da credencial local da própria ferramenta, que a ponte apenas lê e nunca renova; a do Gemini depende da presença e do formato de saída da CLI Antigravity; e a do Codex depende da estrutura dos registros locais. Mudanças nesses formatos exigirão ajustes na ponte. Por isso, cada falha de leitura é sinalizada na interface por um indicador de estado por provedor, em vez de produzir valores incorretos.

Quanto à privacidade, o sistema lê apenas percentuais, horários de reinício e marcas de tempo de atividade. Os prompts, as respostas e as chaves privadas não são lidos, armazenados nem enviados à placa. A única chamada de rede feita pela ponte é a requisição mínima à API da Anthropic, cujo consumo de cota é desprezível.

8. Considerações finais

Este trabalho apresentou o TokenDeck, um sistema de supervisão das cotas de uso de assistentes de IA baseado em ESP32, com comunicação serial USB e aplicação desktop. A solução mostrou que uma placa de baixo custo, mesmo sem PSRAM, é capaz de exibir informações de múltiplas fontes de forma clara e contínua, desde que o processamento pesado fique no computador e o firmware receba apenas dados já normalizados e validados.

Como trabalhos futuros, indicam-se: a medição sistemática da latência entre a mudança de consumo e a atualização da tela; a execução da suíte de testes em integração contínua; o suporte a macOS e Linux na ponte serial; o acréscimo de alertas quando uma janela se aproxima do limite; e a

validação com usuários, a fim de avaliar se o monitor de mesa de fato reduz as consultas manuais e as interrupções por esgotamento de cota.

Referências

ANTHROPIC. **Claude Code: documentação.** Anthropic, 2026. Disponível em: <https://code.claude.com/docs>. Acesso em: 30 out. 2026.

ATZORI, L.; IERA, A.; MORABITO, G. The Internet of Things: a survey. **Computer Networks**, v. 54, n. 15, p. 2787–2805, 2010. DOI: 10.1016/j.comnet.2010.05.010.

BLANCHON, B. **ArduinoJson**: documentação. 2024. Disponível em: <https://arduinojson.org>. Acesso em: 30 out. 2026.

BRAY, T. (ed.). **The JavaScript Object Notation (JSON) Data Interchange Format**. RFC 8259. IETF, 2017. Disponível em: <https://www.rfc-editor.org/rfc/rfc8259>. Acesso em: 30 out. 2026.

ESPRESSIF SYSTEMS. **ESP32 Series Datasheet**. Espressif Systems, 2023. Disponível em: <https://www.espressif.com/en/support/documents/technical-documents>. Acesso em: 30 out. 2026.

GUBBI, J. et al. Internet of Things (IoT): a vision, architectural elements, and future directions. **Future Generation Computer Systems**, v. 29, n. 7, p. 1645–1660, 2013.

ILITEK. **ILI9341: a-Si TFT LCD single chip driver datasheet**. ILI Technology Corp., 2011.

LVGL. **LVGL documentation**. LVGL Kft., 2024. Disponível em: <https://docs.lvgl.io>. Acesso em: 30 out. 2026.

PLATFORMIO. **PlatformIO documentation**. PlatformIO Labs, 2024. Disponível em: <https://docs.platformio.org>. Acesso em: 30 out. 2026.

PYSERIAL. **pySerial documentation**. 2024. Disponível em: <https://pyserial.readthedocs.io>. Acesso em: 30 out. 2026.

PYWEBVIEW. **pywebview documentation**. 2024. Disponível em: <https://pywebview.flowrl.com>. Acesso em: 30 out. 2026.

WEISER, M. The computer for the 21st century. **Scientific American**, v. 265, n. 3, p. 94–104, 1991.